

What is a Constant?

A **Constant** is a fixed value that doesn't change during the execution of program. Once defined, it remains locked in memory.

Definition: A constant is an immutable value that remains completely unchanged throughout the entire lifecycle of a program execution.

Variable vs. Constant

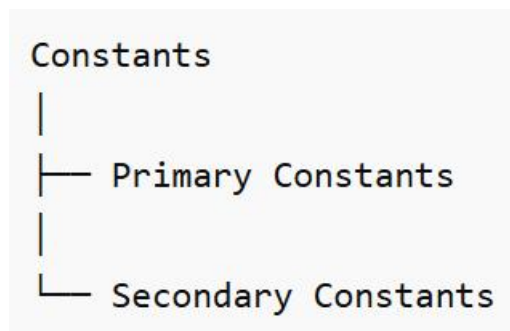
Think of a **Variable** as a cardboard box where you can swap out the contents at any time.

A **Constant** is a glass display case—once an item is placed inside, it is locked and sealed.

Feature	Variable 	Constant 
Value Mutability	Can change during execution	Cannot change during execution
Memory State	Contents vary (age = 20 --> age = 21)	Contents remain strictly fixed
C Code Example	<code>int age = 20;</code>	<code>const float PI = 3.14;</code>
Re-assignment	<code>age = 21; // Allowed</code>	<code>PI = 5.0; //  Compile Error</code>

Classification of Constants

Constants in C are broadly split into two structural groups: **Primary** (the basic building blocks) and **Secondary** (complex structures formed by combining primary constants).



1. Primary Constants

Primary constants represent fundamental, singular data values.

A. Integer Constants

These are whole numbers. They can be positive or negative, but they **must not** contain a decimal point.

Examples: 10, -15, 0, 500

In Code: `int age = 20;` // '20' is the integer constant

B. Real (Floating-Point) Constants

These are numbers that contain fractional parts represented by a decimal point.

Examples: 3.14, -12.35, 0.75

In Code: `float marks = 89.5;` // '89.5' is the real constant

C. Character Constants

A single alphabet, digit, or special symbol enclosed within **single quotes** (' ').

Examples: 'A', '7', '@', '\$'

In Code: `char grade = 'A';` // 'A' is the character constant

2. Secondary Constants

Secondary constants are constructed by combining or referencing primary constants.

A. Array

An array is a collection of similar types of data stored under one name

Code Example: `int marks[5] = {90, 85, 80, 95, 88};`

Memory Visual:

Index:	[0]	[1]	[2]	[3]	[4]
Value:	90	85	80	95	88

Array is considered a secondary constant because it is formed using primary constants.

B. String Constant

A sequence of characters enclosed inside **double quotes** (" "). In memory, C automatically appends a null terminator (`\0`) to signal the end of the string.

Code Example: `char name[] = "Gaurav";`

Memory Visual:



C. Pointer

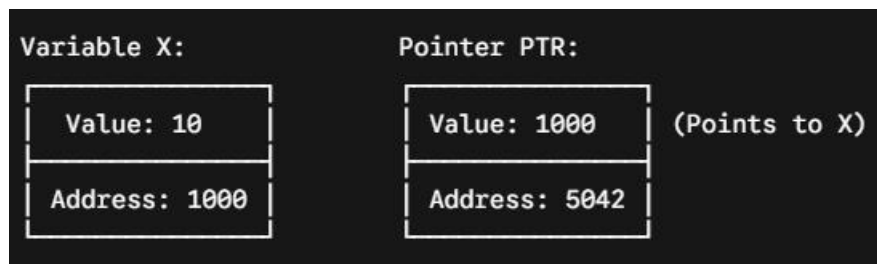
A variable that holds the raw memory address of another variable rather than a literal value.

Code Example:

```
int x = 10;
```

```
int *ptr = &x; // ptr stores the address of x
```

Memory Visual:



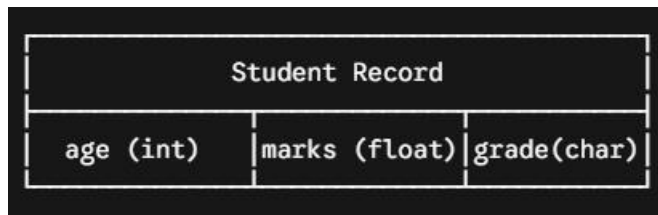
D. Structure (struct)

A user-defined data type that allows you to pack different data types together into a single cohesive block.

Code Example:

```
struct Student {
    int age;
    float marks;
    char grade;
};
```

Memory Visual: Every member gets its own distinct slot in memory:



E. Union

Similar to a structure, but **all members share the exact same memory location**. Only one member can hold a value at any given time.

Code Example:

```
union Data {
    int i;
    float f;
    char c;
};
```

Memory Visual: Members overlap, making unions incredibly memory efficient:



 **Note:** Don't worry if the Secondary Constants look complicated right now. Arrays, Strings, Pointers, Structures, and Unions are deep computational topics that you will explore step-by-step later in your programming journey.